

PPT-LAT-Roadmap

Project: shannon-prime-lattice **Document role:** Operational roadmap. Read by every future session before doing work. **Status:** Living document. Mutable. Papers are scaffolding, not artefacts. **Last rewrite:** 2026-05-21 **Authors:** Knack + Claude + Gemini (Shannon-Prime team)

0. Read-this-first

This document supersedes the prior linear v1 roadmap that staged work as KSTE → engine → multi-node. That framing was too narrow. The project now spans four parallel engine backends, four (and growing) model families, inline weight and KV-cache compression as **foundational** (not an opt-in feature), and the Lattice blockchain track running in parallel with the inference work.

Two binding rules govern every session that picks up this roadmap:

1. **Anti-contamination rule.** Do **not** read, copy, vendor, or reach into source files under D:\F\shannon-prime-repos\shannon-prime\ or D:\F\shannon-prime-repos\shannon-prime-engine\. Those repositories exist for archival reference only. The math papers under D:\F\shannon-prime-repos\papers\PPT-ARM\ are **conceptual reference** — read them for theory, never paste code from them. This project rebuilds everything from scratch. Cross-pollination has already produced regressions in this organization at least ten times in six months. The memory item `feedback_no_cross_contamination` is load-bearing here.
2. **Papers are scaffolding, not artefacts.** PPT-LAT-Theory.md, PPT-LAT-Systems.md, and this file are the lattice from which the system is built, not finished publications. Sessions may amend any of them when reality contradicts the design. The blockchain protocol spec in particular is expected to evolve.

If those two rules feel restrictive, that is the point. The previous sessions that ignored them produced the artefacts collected in the `feedback_no_cross_contamination` and `feedback_dont_frankenpatch` memory items.

1. Why the scope expanded

The original v1 roadmap presumed a single hot path: build KSTE, wire it into one engine, demonstrate dedup, layer blockchain on top. Three realities forced the expansion:

- **Backend reality.** Real users sit on RTX 3000/4000, on Apple Silicon via Vulkan-MoltenVK, on Snapdragon phones with Hexagon DSP and HTP V69, and on Linux servers with AVX-512. A single-backend prototype cannot be promoted to a production system. Each backend has its own build environment, kernel idioms, and bring-up bugs. The roadmap treats them as four parallel tracks because trying to serialize them costs months for no architectural gain.
- **Model-family reality.** Llama 3.x, Qwen3.0–3.7, Gemma 2.5/3/4, and DeepSeek V4 do not share enough surface area to let one model carry the lattice features alone. They differ in RoPE shape, attention grouping, FFN topology (dense vs MoE), normalisation, and tokenisation. The roadmap therefore plans for an explicit model-family × backend matrix (a 7-row × 4-column grid in Phase 3), and assumes that filling this matrix is the project's centre of mass.

- **Compression-foundation reality.** Inline Q8 weight storage with Frobenius scale, Q4 mixed-precision under calibration, and VHT2 + Möbius + Spinor KV-cache compression are not “features that may be enabled later.” They are the **load-bearing memory layout** that makes long-context, multi-model, multi-backend inference fit at all. The roadmap treats compression as bedrock, not topping.

The Lattice features themselves — KSTE dedup, ARM gossip aggregation, dominance verification, two-token economy, blockchain protocol — layer on top of the compressed-inference stack and are off-by-default behind environment-variable gates until they are individually proven.

2. Phase summary table

Phase	Title	Goal in one line	Gate (must pass before advancing)	Wall-clock estimate
0	Bootstrap	Repos, papers, env scripts, workspace	DONE	DONE
1	Math core foundation	O_K, CRT-NTT, poly-ring, VHT2, Frobenius, KSTE built and tested in isolation	All T_* unit tests green on Win+Linux	3–4 weeks
2-CPU	Engine, CPU backend	Reference forward pass + compressed weights + NTT attention on x86	E_CPU_1..E_CPU_6 green	4–6 weeks
2-CU	Engine, CUDA backend	Same scope as 2-CPU on NVIDIA GPU	E_CU_1..E_CU_6 green	4–6 weeks
2-VK	Engine, Vulkan backend	Same scope as 2-CPU on cross-platform GPU	E_VK_1..E_VK_6 green	6–8 weeks
2-HX	Engine, Hexagon backend	Same scope as 2-CPU on Snapdragon HTP V69	E_HX_1..E_HX_6 green	8–10 weeks
3	Model-family expansion	All four backends host all seven model families	M_xB_ matrix green	8–12 weeks
4	Inline cache compression validated	PPL drift and memory savings measured per backend x model validated	Drift $\leq 1\%$ on calibrated families	4 weeks

Phase	Title	Goal in one line	Gate (must pass before advancing)	Wall-clock estimate
5	Lattice features (sieve, ARM, dominance)	Off-by-default ENV-gated overlays	Regression suite green when gates off	6 weeks
6	Two-node CRT-sharded inference demo	Dual-prime forward pass across two boxes	End-to-end demo run	3 weeks
7	DHT crawler skeleton	Kademlia-like routing, single-node first	Two-node lookup works	3 weeks
8	Position-as-Arithmetic crawl assignment	Prime-factored lattice key space drives shard ownership	Deterministic re-assignment proven	2 weeks
9	ARM gossip aggregation	Capacity-bounded HRR merge across peers	Capacity curve plotted	3 weeks
10	Verification layer	Spot-check via $\leq_d$, slashing simulator	Slashing simulator stable	4 weeks
11	Two-token economy simulator	Discovery vs work-token dynamics	Equilibrium observed in simulator	3 weeks
12	Blockchain protocol design	Protocol scaffolding; simulator-only first	Simulator passes 1k-block run	6 weeks
13	End-to-end pilot	3 nodes, full stack, real metrics	Pilot report delivered	4 weeks

Wall-clock estimates assume one solo developer plus AI agents working a standard week. They are **planning numbers**, not commitments. The phase gates are non-negotiable. The estimates are.

2.1 How to read the phase table

A few notes on interpreting the table above for sessions picking up the project cold:

- The phases are not strictly serial. Phase 2's four backends are four parallel tracks. Phase 3's matrix is up to 28 parallel cells. Sections 3 and 20 below detail which work can overlap.

- A **gate** is a binary go/no-go check. If a gate fails, the phase is not closed and no later phase that lists it as a dependency may start. The estimates are not gates and may slip without changing the go/no-go status.
- The phase numbering survives reorganisation. If a future session decides Phase 8 should split into 8a and 8b, the table grows vertically; existing phase numbers do not shift. This is what makes the `lat-phase-<n>-closed` tags durable.

2.2 The big picture in one paragraph

The project rebuilds the Shannon-Prime mathematical machinery from scratch (Phase 1), wires it into four engine backends (Phase 2), spreads it across the in-scope model families (Phase 3), validates inline compression end-to-end (Phase 4), layers the Lattice-specific content-dedup features on top behind environment gates (Phase 5), demonstrates two-machine sharded inference (Phase 6), grows the distributed-system layer (Phases 7–9), adds verification and economy simulation (Phases 10–11), specifies the protocol (Phase 12), and runs a three-node pilot (Phase 13). Every phase has a contract; sessions that close a phase write an offload note and tag the repository.

3. Anti-contamination, contracts, and offload pattern

3.1 Anti-contamination

Every Phase 1 file is created from a blank buffer. The math papers under `papers/PPT-ARM/` may be consulted for theorem statements; their implementations may not be copied. Sessions that violate this rule have historically produced (a) silent format drift between the new repo and the legacy one, (b) duplicate names that confuse later searches, and (c) regressions when a “shared” file in the legacy repo is updated and the new repo silently inherits a behaviour change.

If a session feels the urge to look at the legacy code “just for reference,” the correct move is to read the theorem statement in the paper and re-derive the implementation from scratch. The derivations are short. The bugs that come from cross-contamination are long.

3.2 Contract system

Every phase below lists:

- **Goal** — one paragraph.
- **Deliverables (file list)** — every file the phase must produce.
- **Tests (named, gated)** — every test the phase must pass, named so that future sessions can search for the exact identifier.
- **Entry conditions** — what must be done before starting.
- **Exit conditions** — what must be true to close the phase.
- **Dependencies** — which other phases gate this one.
- **Notes for the picking-up session** — what to read first, what gotchas past sessions have hit.

A phase is closed only when every deliverable exists, every test passes on both Windows and Linux (where applicable), and an offload note has been written.

3.3 Offload pattern

Each working session ends by writing `papers/SESSION-STATE-lat-<phase>.md` with: current phase, files touched, tests passing, tests failing, next concrete action, any open questions.

Sessions that pick up the project read this file first, then this roadmap.

When a phase closes, the corresponding session-state file is renamed to `SESSION-CLOSED-lat-<phase>.md` and a one-paragraph summary is added to this roadmap's "Phase log" appendix.

3.4 VSCode workspace + build environments

The workspace at `D:\F\shannon-prime-repos\shannon-prime-lattice.code-workspace` opens the four repositories the project actually touches:

- `shannon-prime-system` — math primitives.
- `shannon-prime-system-engine` — backend implementations.
- `shannon-prime-lattice` — Lattice features + this roadmap.
- `shannon-prime-lattice-node` — DHT / blockchain node code (later phases).

Build environment scripts live under `shannon-prime-system-engine/scripts/env/`:

- `env-cpu-msvc.bat` — Windows MSVC 19.x, AVX2/AVX512 flags pinned.
- `env-cpu-gcc.sh` — Linux gcc 12+, `-march=native` baseline.
- `env-cuda.bat` — VS2019 Build Tools + CUDA 12.4, `--use-local-env` flag, target SM 75/86/89.
- `env-vulkan.bat` — Vulkan SDK 1.3.x + `glslc`.
- `env-hexagon.bat` — Hexagon SDK 5.x on Windows host, `Git sh.exe` prepended to `PATH` per the `reference_hexagon_build_recipe` memory.

Versions are **pinned** in each script. A session that needs to bump a toolchain bumps the script in a dedicated commit and notes the change in the offload file.

3.5 GitHub workflow

All four repositories are private. Workflow is per-phase push, no pull requests required (solo developer plus agents). Tags are cut at each phase close, named `lat-phase-<n>-closed`. Commits inside a phase use the prefix `[lat-<phase>]` so the log is grep-able.

3.6 Testing discipline

Every phase keeps the **full regression suite green**, not just its own tests. This is the rule that prevents Phase 5+ Lattice features from silently breaking Phase 2 backends. Sessions land work behind a flag if they cannot make the regressions hold; turning the flag on is a separate commit that runs the suite again.

4. The math primitives — what we are recreating

This section names every math primitive the project will rebuild from scratch and what each one is for. Implementations come in Phase 1.

4.1 $\mathcal{O}_K$ integer arithmetic over $\mathbb{Q}(\sqrt{-163})$

$\mathbb{Q}(\sqrt{-163})$ is one of the nine imaginary quadratic fields with class number 1 (a Heegner number). The ring of integers $\mathcal{O}_K$ is a unique factorisation domain. We choose $\mathbb{Q}(\sqrt{-163})$ specifically because:

- UFD means every nonzero non-unit factors uniquely (up to units and order) into irreducibles. That property anchors Möbius reconstruction and lets us treat token indices as products of irreducibles without ambiguity.

- The class group is trivial. No nontrivial ideal class equivalences haunt the storage layout.
- The discriminant -163 fits comfortably in 64-bit arithmetic for the norms we care about.

All exact arithmetic — KSTE node values, Möbius reconstructions, ARM binding accumulators — happens in O_K . The implementation must support add, mul, conjugate, norm, division-with-remainder when the divisor's norm permits, and irreducible factorisation up to a configurable norm bound.

4.2 Polynomial-ring attention over $R_q = \mathbb{Z}_q[x]/(x^N+1)$

R_q is the negacyclic polynomial ring used by CKKS and most modern lattice cryptosystems. We use it as a **replacement for the softmax attention kernel**: queries, keys, and values are encoded as polynomials in R_q ; the inner-product step becomes a polynomial multiplication modulo $(x^N + 1)$; the softmax becomes a p-adic exponential on the coefficients followed by a normalisation pass.

Phase 1 targets $N = 256$ (matching the per-head dimension of the current model targets) and parameterises N over $\{128, 256, 512, 1024\}$ so that all four sizes share a single code path.

4.3 CRT-NTT dual-prime kernel with no `__int128`

Multiplying in R_q efficiently requires the Number Theoretic Transform. We use a dual-prime CRT decomposition so the kernel never needs 128-bit arithmetic and ports to any 64-bit ALU including Hexagon and Vulkan compute shaders.

- $q_1 = 1073738753$, $q_2 = 1073732609$ (two 30-bit Proth primes).
- Each prime has a primitive $2N$ -th root of unity for N up to 1024.
- The combined modulus $M = q_1 \cdot q_2 \approx 2^{60}$ carries the full result without overflow.
- CRT recombination uses Barrett reduction; no 128-bit divide.

Independent NTT branches run per prime. Bit-exactness is checked against a reference 60-bit `__int128` implementation that exists **only as a parity oracle** (compiled out of production builds).

4.4 Frobenius lifting for Q8 / Q4 weight storage

Frobenius lifting takes a quantised weight stored as a small integer plus a per-row shift and decompresses it inline at matmul time. The per-row Frobenius scale is the key design choice: a single per-tensor shift collapses to noise on Q4 (the cautionary tale from prior validation), while per-row shifts hold accuracy.

- Q8: 1 byte per coefficient + per-row scale. 8× memory compression of the unquantised arena. Production target.
- Q4: 4 bits per coefficient + per-row scale + calibration table. Mixed-precision path; gated on a per-tensor calibration that may promote outlier rows to Q8.

Decompression is fused into the matmul kernels, not done as a separate pass. There is no decoded scratch arena in production builds.

4.5 VHT2 + Möbius reorder + 63-byte Spinor block

The KV cache is stored as a sequence of 63-byte Spinor blocks. Each block carries:

- A VHT2 (Vilenkin Hierarchical Transform) header that describes which basis vectors the block compresses.
- A Möbius-reordered coefficient body.
- A trailing checksum byte.

The byte layout is **frozen** the moment Phase 1D ships. Future sessions do not adjust it without a compatibility migration plan. The cache file format and the on-wire DHT format both reference this layout, so a silent change breaks every node on the network simultaneously.

4.6 KSTE encoder + Tier-0/Tier-1 dominance

The Knight-Spinor Tree Encoder maps a K-vector into a 64-byte packed tree in $T_{\{60,3\}}$. The encoder is deterministic, lossless under the dominance partial order, and produces a compact signature suitable for hashing.

Tier-0 dominance compares roots. Tier-1 dominance compares the first level of children. The Lattice dedup mechanism uses Tier-0 to bucket content into rough equivalence classes and Tier-1 to confirm a match before counting it as a duplicate. The encoder must produce identical output on every backend so that two nodes can compare signatures without a tie-break protocol.

4.7 ARM — Algebraic Resonance Memory

ARM stores HRR-style key-value bindings in the CRT cyclotomic ring. Binding is a polynomial multiplication; recall is an inverse-binding multiplication followed by a similarity search. The choice of the CRT cyclotomic ring is what lets ARM share its kernel inventory with the poly-ring attention layer: a single NTT-based pointwise multiply serves both. Storing ARM bindings as 63-byte Spinor blocks would in principle work too; in practice we store the polynomial coefficients directly because ARM banks are written-once-read-many and benefit from a flat layout.

Phase 1 ships ARM as a single-node primitive. Phase 9 layers gossip on top so peers can merge their ARM banks with bounded capacity loss. The capacity bound is the usual HRR $\sqrt{K}$ bound: with K bindings packed into a single ring element of dimension N , recall accuracy degrades as roughly $\sqrt{K/N}$. For $N = 256$ this means a slab holds about 16 high-confidence bindings before recall fidelity drops below 0.9 cosine similarity, which is the threshold the Phase 9 capacity-curve experiment will report against.

4.8 Inline weight compression as a foundation, not a feature

The single most consequential design choice in this roadmap is that inline Q8 (and eventually Q4) weight storage is not an optional optimisation but the **default memory layout**. Every backend’s matmul kernel reads packed bytes and applies the per-row Frobenius scale inline; there is no decoded fp32 arena that holds the same weights in expanded form. The reason this matters is twofold:

- Memory pressure. An unquantised arena on Gemma3-1B already costs 10.4 GB; the Q8 packed form is 1.3 GB. The unquantised arena does not fit on the phone path at all, and it costs 8× the network transfer on the multi-node CRT-shard path.
- Bandwidth determinism. Every backend uses the same packed byte layout, so cross-backend verification compares bytes for bytes rather than fp32 approximations. This is what makes the cross-backend determinism test (OQ-8) tractable at all.

The cost of treating compression as foundational is that every backend must implement the packed-byte matmul before any larger model can be brought up. Sessions that attempt to “do correctness first, compression later” hit a wall when the unquantised path runs out of memory on the test rigs and need to backtrack to compression-first anyway.

4.9 Inline KV-cache compression as a foundation

The same logic applies to the KV cache. The Spinor block layout specified in §4.5 is the only KV layout in the production paths. There is no “uncompressed KV cache” mode in production

builds; the reference fp32 cache exists for the parity tests only, compiled out of release builds. The reason is identical to the weight-compression case: long-context inference does not fit on any of the target backends without KV compression, and the CRT-shard and DHT paths both consume the on-disk and on-wire byte layout directly, so the production format has to match from day one.

5. The 13-step Prime Power Transformer (canonical table)

The replacements below are the canonical algebraic interpretation of every step in a transformer forward pass under the PPT framework. They are reproduced verbatim from Paper I §10 so future sessions can grep for the exact wording.

Step	Operation	Algebraic replacement
1	Embedding lookup	Möbius reconstruction over squarefree token indices; CRT vocabulary sharding
2	RMSNorm (pre-attn)	Mersenne-prime scaling; Poncelet closure $d^2=R^2-2Rr$
3	Q/K/V projections	Twin-prime head pairing; sexy-prime 6:1 GQA grouping
4	SP Write (KV → archive)	Poncelet closure as eviction trigger; CRT-sharded KV
5	FUSED_KQ	UFD-exact decompression; Heegner endomorphism
6	Softmax	p-adic exponential on integers; circulant attention on closed orbits
7	Fused V weighted sum	Spinor reconstruction across twin-paired heads
8	Attention output projection	CRT decomposition of W_O into independent sub-matrices
9	FFN (skeleton + residual)	Mersenne-dimensional skeletons; n^2+n+41 cold-start
10	Activation oracle update	Cramér prime-gap prefetch; Poncelet early exit
11	Residual add + norm	Group-law residual on $E(K)$
12	Per-layer loop	$n\delta\equiv 0$ adaptive depth; caustic projection
13	LM head	CRT pruning of vocabulary logits; Mersenne-prime sampling

This is the operational mapping from “what the model does” to “what we build.” Every Phase 2 backend ultimately wires these thirteen steps into its forward pass. Every Phase 3 model family threads the same table through its specific topology.

6. Theorems and extensions on the anchor

Treat the following as the “already proven” set. Sessions do not need to re-prove them; they need to call out when an implementation contradicts them.

- **T1 — Endomorphism realization.** A hidden-state trajectory through L transformer layers embeds in the L -fold product of an elliptic curve E over O_K . Layer composition is endomorphism composition.

- **T2 — Möbius UFD compression.** Exact reconstruction over O_K is possible at the squarefree basis. This is the theoretical anchor for the squarefree token index in step 1.
- **T3 — Hasse-Weil = Shannon limit.** The Hasse-Weil bound $|E_p(F_p) - (p+1)| \leq 2\sqrt{p}$ coincides with the Shannon-channel capacity bound applied to the CM curve. The two limits are the same constant to within a unit.
- **T4 — Frobenius cancellation.** On Gemma3-1B, the Frobenius lift is bit-identical to six significant figures. The validation produced PPL 13.11 against a baseline of 13.12 (delta 0.08%). This number is the acceptance criterion for any new model under the same Frobenius configuration.
- **T5 — Deuring / CM Sato-Tate.** The Frobenius trace a_p is asymmetrically distributed between primes that split and primes that remain inert in O_K . This asymmetry is what makes class-number-1 fields work for compression — the inert primes carry more bits per symbol.
- **T6 — CRT exact sharding.** The dual-prime kernel is bit-identical to the 60-bit reference on Linux gcc and Windows MSVC. Portability across any 64-bit ALU is implied.
- **E9.1 — Stern-Brocot RoPE.** Discrepancy $\phi = 0.00134$ against 0.05576 for standard RoPE. The Stern-Brocot positional encoding is a strict improvement on quasirandomness grounds.
- **E9.2 — Weil pairing on $E[n]$.** Miller’s algorithm validated; bilinearity confirmed empirically. The pairing math is solid; the open research question is the embedding from hidden states into $E[n]$.
- **E9.3 — Hecke multiplicativity.** 20 of 20 random trials confirm the Hecke operator multiplicativity at composite levels.
- **E9.5 — LLL reduction.** 20 of 20 trials confirm LLL-reduced bases produce smaller KV-write footprints than the naive basis.
- **E9.6 — BSD analytic rank.** Toy curves verified via Sage. Not load-bearing for the production system; included for completeness.
- **E10 — Iwasawa $\mu=0$.** Residual-stream depth stability follows from the $\mu=0$ conjecture in the relevant Z_p -extension. The empirical evidence is consistent with $\mu=0$ on all curves tested.

When a Phase 2 backend produces a result inconsistent with one of these theorems, the theorem is correct and the implementation is wrong. That is the working assumption until proven otherwise.

The reason this rule reads so strongly is operational: every time a prior session has assumed the math was off, the math has turned out to be right and the bug has been in the storage layout, the kernel ordering, or a sign convention. The theorems above have been validated to six significant figures on real hardware running real models; the chance that a fresh backend implementation is the first place those theorems fail is vanishingly small compared to the chance that the implementation has a transposed index somewhere. The corollary is that when a backend disagrees with another, the older backend is the ground truth and the newer one is the suspect.

A second corollary: do not bend the math to make the implementation easier. There is a recurring temptation, especially on the Phase 2-HX track, to drop the dual-prime CRT in favour of a single 30-bit prime because “the test corpus fits anyway.” Resisting that temptation is the whole reason Phase 1B’s `T_NTT_5` grep gate exists. The dual-prime kernel is not an optimisation. It is the substrate Phase 6’s two-node demo runs on, the substrate Phase 8’s position-as-arithmetic key derivation reads from, and the substrate the verification layer audits. Collapsing it to single-prime to save bring-up time costs multiple later phases.

7. Phase 1 — Math core foundations

Goal. Stand up every math primitive in section 4 as a standalone library inside shannon-prime-system, with deterministic unit tests that run on Windows MSVC and Linux gcc.

Dependencies. Phase 0 (DONE).

Subphases run in parallel. Phase 1A through 1F are independent modules. A session can pick any one without blocking the others. The recommended priority order — for a serial reader — is 1A → 1B → 1D → 1C → 1E → 1F, because the polynomial-ring and Frobenius layers benefit from the CRT-NTT kernel being already in place, and KSTE wants the VHT2 block format frozen first.

Why these particular primitives. The Phase 1 list is not a buffet of mathematical curiosities. Each primitive answers a specific question about how the production system stores and moves bits:

- **O_K** answers “how do we factor token indices uniquely so Möbius reconstruction works without ambiguity?” The class-number-1 property is the load-bearing piece.
- The dual-prime CRT-NTT answers “how do we multiply in the cyclotomic ring on every backend without needing 128-bit arithmetic?”
- Polynomial-ring attention answers “how do we replace softmax with something that lives in the same algebraic universe as the cache representation, so the kernel inventory shrinks?”
- The Spinor block answers “what is the on-disk and on-wire byte layout of a single KV record?” The 63-byte frozen layout is the contract.
- Frobenius lifting answers “how do we store weights at 1 byte per coefficient and still get bit-identical PPL?”
- KSTE answers “how do we produce a content signature that two nodes can compare bit-for-bit without a tie-break protocol?”

Implementing all six in parallel is what makes Phase 1 fit in a few weeks rather than a few months. Sessions that try to serialise will discover that the bottleneck is rarely arithmetic — it is build environments, harness wiring, and offload notes. Spreading the implementation work across parallel subphases keeps any single bottleneck from gating the whole phase.

7.1 Phase 1A — O_K arithmetic over $\mathbb{Q}(\sqrt{-163})$

- **Deliverables.** `system/ok/ok_int.h`, `ok_int.c`, `ok_int_test.c`.
- **Tests.**
 - `T_OK_1` — UFD verification on 256 random norms $\leq 2^{20}$.
 - `T_OK_2` — Norm and conjugate identities ($N(\alpha\beta) = N(\alpha)N(\beta)$, $\text{conj}(\text{conj}(\alpha)) = \alpha$).
 - `T_OK_3` — Multiplication closure (every product lands back in $\mathbb{O}_K$ with correct integer coordinates).
 - `T_OK_4` — Ideal class triviality: every fractional ideal of norm $\leq 2^{16}$ is principal.
 - `T_OK_5` — Round-trip from rational integer to $\mathbb{O}_K$ and back.
 - `T_OK_6` — Irreducible factorisation for 1024 random norms $\leq 2^{14}$.
- **Entry conditions.** None.
- **Exit conditions.** All six tests pass on Win+Linux. No undefined behaviour under UBSan.
- **Notes.** The Heegner number 163 is non-negotiable. Other Heegner numbers exist (1, 2, 3, 7, 11, 19, 43, 67, 163) but only 163 has the discriminant we want.

7.2 Phase 1B — CRT-NTT dual-prime kernel

- **Deliverables.** `system/ntt/ntt_crt.h`, `ntt_crt.c`, `ntt_crt_test.c`, `ntt_ref_int128.c` (parity oracle, compiled out of production).

- **Tests.**
 - T_NTT_1 — Forward/inverse round-trip on 4096 random polynomials at each $N \in \{128, 256, 512, 1024\}$.
 - T_NTT_2 — Bit-exactness vs `ntt_ref_int128` on Linux gcc.
 - T_NTT_3 — Bit-exactness vs `ntt_ref_int128` on Windows MSVC.
 - T_NTT_4 — Pointwise multiply followed by inverse equals coefficient-wise modular multiplication of the original polynomials (negacyclic convolution).
 - T_NTT_5 — No 128-bit type used in the production path (static assert in header, plus a grep gate in CI).
- **Entry conditions.** None.
- **Exit conditions.** All five tests pass.
- **Notes.** Barrett reduction for the CRT step. Twiddle factors precomputed and cached per N . The parity oracle is a regression anchor for the lifetime of the project — keep it building even when it is not in the production path.

7.3 Phase 1C — Polynomial-ring attention R_q

- **Deliverables.** `system/poly/poly_ring.h`, `poly_ring.c`, `poly_ring_test.c`.
- **Tests.**
 - T_PR_1 — Polynomial multiply matches schoolbook reference at $N \in \{128, 256, 512, 1024\}$ for 1024 random polynomial pairs each.
 - T_PR_2 — Inner-product attention $KL \leq 1e-7$ versus softmax baseline at $d = 256$ (matches the prior Gemma3 head-size result on the legacy repo).
 - T_PR_3 — Negacyclic property: $x \cdot p(x)$ wraps as expected (the last coefficient becomes the negated leading coefficient).
 - T_PR_4 — Cross- N stability: the same input expressed at $N=256$ and $N=512$ (zero-padded) produces the same result up to the shared coefficient prefix.
- **Entry conditions.** Phase 1B closed.
- **Exit conditions.** All four tests pass.
- **Notes.** The KL bound T_PR_2 is the closest thing the project has to a soft acceptance criterion for “the math is right.” Take it seriously.

7.4 Phase 1D — VHT2 + Möbius reorder + 63-byte Spinor block

- **Deliverables.** `system/spinor/spinor_block.h`, `spinor_block.c`, `vht2.c`, `möbius_reorder.c`, `spinor_test.c`. Frozen byte layout specified in `papers/PPT-LAT-Systems.md §3`.
- **Tests.**
 - T_VHT_1 — Block round-trip: encode then decode identical bytes for 65,536 random vectors.
 - T_VHT_2 — Möbius reorder bijection: every permutation is a true permutation (no element collisions).
 - T_VHT_3 — 63-byte total size verified by `sizeof(spinor_block_t)`.
 - T_VHT_4 — Checksum byte detects any 1-bit corruption in the body.
 - T_VHT_5 — Endianness: identical bytes produced on little-endian Linux and Windows hosts.
 - T_VHT_6 — Layout frozen flag: a `LAYOUT_VERSION` constant in the header must be incremented by hand if any field moves, and CI refuses a layout change without a migration plan file.
- **Entry conditions.** None.
- **Exit conditions.** All six tests pass; layout reviewed and signed off in the offload note.
- **Notes.** The 63-byte total is deliberate — 64 minus the checksum, packed tight, no padding. Future sessions: do not add a “convenient” reserved byte.

7.5 Phase 1E — Frobenius lift for Q8 weight storage

- **Deliverables.** `system/frobenius/frobenius_lift.h`, `frobenius_lift.c`, `frobenius_test.c`.
- **Tests.**
 - T_FRO_1 — Per-row scale picker correctness against the ceiling-shift reference (which exists from prior validation; reimplemented from scratch under the anti-contamination rule).
 - T_FRO_2 — Encode-then-decode round-trip max error ≤ 1 ULP for a fixed test matrix.
 - T_FRO_3 — 8× memory reduction confirmed on a 4096×4096 weight matrix.
 - T_FRO_4 — On Gemma3-1B, Frobenius-lifted weights produce PPL within 0.1% of baseline. This is the T4 acceptance check repeated under new code.
- **Entry conditions.** Phase 1A closed.
- **Exit conditions.** All four tests pass; T_FRO_4 logged as the reference number for future drift checks.
- **Notes.** Q4 is **not** in Phase 1E. Q4 lives in Phase 4 because it needs calibration tables that depend on a working forward pass.

7.6 Phase 1F — KSTE encoder + Tier-0/Tier-1 dominance

- **Deliverables.** `system/kste/kste_encode.h`, `kste_encode.c`, `kste_dominance.c`, `kste_test.c`.
- **Tests.**
 - T_KSTE_1 — Encoder determinism: identical K-vector produces identical bytes 1,000,000 trials.
 - T_KSTE_2 — Tier-0 dominance partial-order axioms: reflexivity, antisymmetry up to equivalence, transitivity.
 - T_KSTE_3 — Tier-1 confirmation rule: if two trees pass Tier-0 and Tier-1, they share the same first-level child set.
 - T_KSTE_4 — Cross-platform identity: encoder produces identical bytes on Windows MSVC and Linux gcc.
 - T_KSTE_5 — 64-byte packed-tree size enforced.
- **Entry conditions.** None.
- **Exit conditions.** All five tests pass.
- **Notes.** Tier-0/Tier-1 nomenclature comes from the original KSTE paper. Future sessions sometimes try to “simplify” to a single tier for ergonomic reasons; the two-tier check is what makes the Lattice dedup adversarially robust. Do not collapse it.

7.7 Phase 1 exit

Phase 1 closes when all six subphases close, the regression runner prints six green sections, and `SESSION-CLOSED-lat-1.md` is written. The repository tag is `lat-phase-1-closed`.

7.8 Pseudocode sketch — Phase 1B kernel skeleton

The following pseudocode captures the shape of the dual-prime CRT-NTT kernel without descending into a real implementation. It exists here to anchor the API a Phase 1B session is expected to land.

```
struct ntt_ctx {
    uint32_t N;           // ring degree, in {128, 256, 512, 1024}
    uint32_t q1, q2;      // two 30-bit Proth primes
    uint32_t* psi1; uint32_t* psi2; // 2N-th roots of unity per prime
    uint32_t* inv_psi1; uint32_t* inv_psi2;
```

```

    // Barrett mu values for each prime
};

ntt_ctx* ntt_init(uint32_t N);
void ntt_forward(const ntt_ctx*, const int32_t* in, uint32_t* out1, uint32_t* out2);
void ntt_inverse(const ntt_ctx*, const uint32_t* in1, const uint32_t* in2, int64_t* out);
void ntt_pointwise_mul(const ntt_ctx*, const uint32_t* a1, const uint32_t* a2,
                        const uint32_t* b1, const uint32_t* b2,
                        uint32_t* out1, uint32_t* out2);
void ntt_crt_recombine(const ntt_ctx*, const uint32_t* x1, const uint32_t* x2, int64_t* out);

```

The contract is: `ntt_forward` produces two residue vectors; pointwise multiply happens per residue (no cross-prime arithmetic); `ntt_inverse` applies the inverse transform per residue and `ntt_crt_recombine` reassembles a signed 60-bit result via Barrett reduction. No 128-bit type appears anywhere. The same kernel is reused by 1C (poly-ring attention), 4.7 (ARM), and 1E (Frobenius matmul) without modification.

7.9 Pseudocode sketch — Phase 1D Spinor block

```

#define LAYOUT_VERSION 1u // bumping requires migration plan
struct spinor_block_t { // 63 bytes total, packed
    uint8_t vht2_header[7];
    uint8_t mobius_body[55];
    uint8_t checksum; // CRC-8 of header || body
};
_Static_assert(sizeof(spinor_block_t) == 63, "frozen layout");

```

Encoders and decoders must round-trip on every backend; the cross-platform identity test `T_VHT_5` specifically compares bytes byte for byte between Linux gcc and Windows MSVC builds.

8. Phase 2 — Engine backend tracks (CPU, CUDA, Vulkan, Hexagon)

Goal. Each backend supports a single reference model end-to-end: GGUF load → forward pass → token-level output. Frobenius-lifted weights are inline-decompressed. NTT-attention is wired but the **sieve is OFF**. Inline KV-cache compression is wired but the **Lattice features are OFF**. The reference model is Qwen3-0.6B Q8.

Dependencies. Phase 1 closed.

Parallelism. The four backends are independent. Sessions can run 2-CPU and 2-CU concurrently with no shared state. 2-VK and 2-HX can start the moment their build environments are sorted, regardless of 2-CPU and 2-CU status. The matrix below shows the per-backend subphase plan — same letter-coded structure across all four.

8.1 Per-backend subphase template

For each backend $B \in \{\text{CPU, CU, VK, HX}\}$:

- **2-B.A — GGUF loader.** Parse the GGUF header, materialise tensors into backend-native storage. No SP transforms applied yet.
- **2-B.B — Forward pass on Qwen3-0.6B Q8.** Reference correctness vs llama.cpp logits on the same prompt. Acceptance: max-token logit difference $\leq 1e-4$ absolute or $\leq 0.1\%$ relative.

- **2-B.C — Frobenius-quantised matmul with inline decompression.** Production memory layout. 8× compression confirmed end-to-end.
- **2-B.D — Backend-specific vectorisation.** AVX2 for CPU, AVX512 optional second pass, CUDA tensor cores where applicable, Vulkan subgroup ops, HVX vectorisation for Hexagon.
- **2-B.E — NTT-attention path wired in.** Sieve OFF. PPL must remain within Phase 1C's T_PR_2 tolerance.
- **2-B.F — KSTE-encoded KV cache.** Gated by SP_KSTE_KV=1. Off by default. Phase 2 only requires that the encode path produces identical signatures across backends and that decode reproduces the same KV.

Tests E_B_1..E_B_6 mirror the six subphases. The numbering follows the subphase letter: E_CPU_1 covers 2-CPU.A, E_CPU_2 covers 2-CPU.B, and so on.

8.2 Phase 2-CPU (the canonical track)

- **Build env.** scripts/env/env-cpu-msvc.bat (Windows) and scripts/env/env-cpu-gcc.sh (Linux).
- **Reference model.** Qwen3-0.6B Q8.
- **Tests.**
 - E_CPU_1 — GGUF loader round-trips header bytes.
 - E_CPU_2 — Forward pass within tolerance of llama.cpp on first 256 tokens of the Pile sample.
 - E_CPU_3 — Frobenius matmul produces identical logits to a reference fp32 matmul on the same prompt.
 - E_CPU_4 — AVX2 path matches scalar within 1e-6 elementwise; AVX512 path optional but if present matches AVX2 within 1e-6.
 - E_CPU_5 — NTT-attention end-to-end PPL within T_PR_2 of softmax baseline. Sieve OFF.
 - E_CPU_6 — KSTE KV-cache encode/decode round-trip identical bytes. Sieve OFF; this only exercises the encoder.
- **Notes for picking up.** CPU is the canonical track because it has the fewest build dependencies. Bring up CPU first if any other backend is blocked; using CPU outputs as the ground truth for the other backends is built into the test harness.

8.3 Phase 2-CU (CUDA backend)

- **Build env.** scripts/env/env-cuda.bat. Pinned to CUDA 12.4 + VS2019 Build Tools because that combination has the longest stability record on this team's hardware (reference_cuda_build_recipe memory).
- **Reference model.** Same Qwen3-0.6B Q8.
- **Tests E_CU_1..E_CU_6** mirror the CPU set, with the additional per-platform gate that CUDA output must match CPU output within 1e-3 relative on the same prompt. The looser tolerance accounts for expected fused-multiply-add ordering differences.
- **Notes.** Target SM 75/86/89 (RTX 2060/3000/4000 series). Older SMs not supported. Use --use-local-env to keep MSVC from injecting unwanted flags.

8.4 Phase 2-VK (Vulkan backend)

- **Build env.** scripts/env/env-vulkan.bat. Vulkan SDK 1.3.x. glslc for shader compilation.
- **Reference model.** Same Qwen3-0.6B Q8.
- **Tests E_VK_1..E_VK_6** mirror the CPU set, with output vs CPU matching within 1e-3 relative.

- **Notes.** Lower initial priority than CPU/CUDA. The reason Vulkan is in scope at all is Apple Silicon support via MoltenVK and the option of a cross-platform fallback when CUDA is not available. Subgroup ops are essential — the reduction-heavy parts of NTT-attention rely on them.

8.5 Phase 2-HX (Hexagon backend)

- **Build env.** `scripts/env/env-hexagon.bat`. Hexagon SDK 5.x on the Knack Windows host. `Git sh.exe` prepended to `PATH`. The `reference_hexagon_build_recipe` memory captures the exact recipe; diverging from it has produced multi-session bring-up delays.
- **Reference model.** Same Qwen3-0.6B Q8, with a phone-suitable context length cap.
- **Tests E_HX_1..E_HX_6** mirror the CPU set, with output vs CPU matching within 1e-3 relative.
- **Notes.** This is the highest build-environment-fragility track in Phase 2. Two prior sessions have hit silent fallback bugs when FastRPC `rpcmem` registration size mismatched the IDL length parameter (`feedback_fastrpc_exact_alloc`). Do not exceed-allocate. Use the `freethedsp` shim when `SP_FREETHEDSP=1`; it is opt-in. QNN HTP V69 is the target accelerator for matmul on this platform.

8.6 Phase 2 exit

Phase 2 closes when at least one backend (CPU is the minimum) has all six E-tests green. Other backends close independently and are tagged `lat-phase-2-<backend>-closed`. The CPU backend close also produces `lat-phase-2-closed` since CPU is the canonical anchor.

8.7 Notes for the picking-up session (per backend)

CPU. Start from a blank `engine/cpu/forward.c`. Wire the GGUF loader first, get tensors materialised into row-major-by-token layout, then implement the 13 steps as 13 functions. Keep the AVX2 path beside a scalar reference under an `SP_CPU_SCALAR=1` env gate so the scalar path stays buildable. The AVX512 path is a second pass and not required to close `E_CPU_4`.

CUDA. Mirror the CPU layer-by-layer. The most fragile cell is the fused matmul + Frobenius decompress kernel — get the scalar correctness right via `Ninja -use-local-env` before turning on cublas LT or any tensor-core fusion. Output verification compares against the CPU run for the same prompt, so keep the CPU build alive in the same Phase 2-CU sessions.

Vulkan. Compile glslc shaders into SPIR-V at build time, not at runtime; the JIT path has caused at least one driver crash in prior attempts. Subgroup ops vary by vendor; test under at least two vendors before declaring 2-VK closed. MoltenVK on Apple Silicon is the canonical “third vendor” the test harness picks up later.

Hexagon. Build environment first. Do not attempt to bring up the forward pass until `env-hexagon.bat` produces a working `qaic.exe` invocation per the `reference_hexagon_build_recipe` memory. The FastRPC silent-fallback bug (`project_hexagon_silent_fallback`) is load-bearing context — when in doubt, log every `rpcmem` registration size and grep the logs against the IDL parameter sizes.

9. Phase 3 — Model-family expansion

Goal. Every backend supports every model family in the in-scope list. The matrix is large (7 model families × 4 backends = 28 cells), and not every cell must close — some cells are explicit “later” cells (see priority below).

Dependencies. Phase 2-CPU closed. Other backends may or may not be closed; the matrix is filled per-cell.

Parallelism. Cells are independent. Sessions can pick any cell.

9.1 Model-family list

- 3.1 Llama 3.1 / 3.2 — established baseline; Llama 3 is the most-tested upstream architecture, so it exercises every loader edge case.
- 3.2 Qwen3 base — closest to the current canonical Qwen3-0.6B test model.
- 3.3 Qwen3.5 — incremental update on Qwen3, mostly architecture-flag differences.
- 3.4 Qwen3.6 — **MoE**. Adds the routing layer, sparse FFN, expert parameter sharding. Largest single-cell scope in Phase 3.
- 3.5 Qwen3.7 — incremental update on 3.6 if it lands by then; otherwise deferred.
- 3.6 Gemma 2.5 / 3 / 4 — Google family. Different RoPE shape, different RMSNorm placement, attention sliding window. Gemma 3 is the canonical research target (the T4 validation curve was on Gemma3-1B).
- 3.7 DeepSeek V4 — large MoE with FP8 weights. Lower priority for initial bring-up; the architecture is heavy and the FP8 weights would require an additional dequant path.

9.2 Priority cells

The matrix has 28 cells. To stay honest about scope, the project declares which cells **must** close by end of Phase 3 and which are later-phase:

- **Must close (8 cells).** CPU × {Llama 3.x, Qwen3, Gemma 3}, CUDA × {Llama 3.x, Qwen3, Gemma 3}, Hexagon × {Qwen3, Gemma 3}.
- **Should close (10 cells).** CPU × {Qwen3.5, Qwen3.6, Gemma 2.5, Gemma 4, DeepSeek V4}, CUDA × {Qwen3.5, Qwen3.6, Gemma 2.5, Gemma 4, DeepSeek V4}.
- **Later (10 cells).** All Vulkan cells beyond the canonical Qwen3-0.6B test model. All Hexagon cells beyond Qwen3 and Gemma 3.

9.3 Per-cell deliverables

Each cell delivers:

- engine/models/<family>/loader.<backend>.c — model-specific loader.
- engine/models/<family>/forward.<backend>.c — forward pass.
- engine/models/<family>/test_<backend>.c — correctness tests.
- An entry in engine/models/matrix_status.md updated when the cell closes.

9.4 Per-cell tests

- M__1 — model loads.
- M__2 — first 256 tokens of forward pass match llama.cpp within 1e-3 relative.
- M__3 — PPL on a standard 4k-token sample within 0.5% of baseline.
- M__4 — memory footprint reported in the offload note.

Cells close on M_4 passing. A failed M_2 indicates a loader or forward-pass bug; do not declare the cell closed by skipping it.

9.5 Phase 3 exit

Phase 3 closes when the 8 must-close cells are green and the should-close cells are at least started. Tag lat-phase-3-closed. Later cells continue to land in Phase 4+ alongside their own work.

9.6 Architectural notes per model family

Llama 3.1 / 3.2. Most-tested upstream architecture, which means the loader exercises every GGUF metadata edge case the project will encounter. Use Llama 3.x as the canary when adding any new loader field. Llama’s RoPE is the “plain” reference shape; everything else in the family list is a delta against it.

Qwen3 base. Closest to the canonical test model and the easiest cell to bring up second. The Qwen3 RoPE includes a per-head linear bias which the Stern-Brocot variant (E9.1) wires into directly; keep both code paths buildable until Phase 4 has Stern-Brocot validated end-to-end on this family.

Qwen3.5. Incremental delta on Qwen3 base. Most of the bring-up cost is metadata flags and a slightly different normalisation order inside the attention block; the kernel inventory is identical.

Qwen3.6 (MoE). This is the architectural step change inside the Qwen family. The routing layer assigns each token to k-of-N experts and the FFN becomes a sparse gather. The cell must add an expert parameter sharding path so the experts can be split across multiple machines or pipeline stages later. Phase 3 only requires the single-machine case; multi-machine MoE is a later phase.

Qwen3.7. Speculative. If 3.7 has shipped by the time the project gets here, treat it as another incremental Qwen update; if it hasn’t, defer this row.

Gemma 3 (and 2.5 / 4). Different RoPE shape, different RMSNorm placement (pre + post on some sublayers), attention sliding window. Gemma 3 is the canonical research tar